EECS 348 Assignment 4: Comparing Two AI-Generated Sudoku Solvers

Francis Tryban · EECS 348 · October 15, 2026

For this assignment, I gave two GenAIs the same C++ Sudoku problem, tested what they made, and then picked one to improve. The generated code below is included exactly as the models wrote it. I used Claude Opus 5.5 to write the test scripts, to type my two prompts into the Codex app for the Codex run, and to turn the test results into an earlier draft of this analysis. GPT-6 Sol in the Codex app helped me edit this analysis into my own words, and Claude Opus 5.5 put the timing numbers back into my final version and drafted 2f and 2g from my notes. My comparison is based on the programs and the saved test results in my working folder.

2a–2b. The two GenAIs

I used **Claude Haiku 4.5** through Claude Code in my Mac's terminal and **GPT-6 Luna** through the Codex desktop app on the same Mac. Each one worked in its own folder. Codex was on its Light effort setting. I used the same two small models for Assignment 3, so this lets me see how they do on a harder problem.

2c. The prompt

I put the same program instructions in both `Claude/PROMPT.md` and `Codex/PROMPT.md` and checked that the two files match exactly. The instructions are the program part of the assignment, copied word for word, plus a short section saying how the program gets its puzzle files. The five puzzle files and `solution1.txt` were in each folder. Both `AGENT.md` files were empty. I gave each model the same instruction: read `AGENT.md` and `PROMPT.md`, write the C++ program, and save it as `iterations/00-raw.cpp`. Neither model saw the rubric or the other model's code.

Here is the full prompt both models received:

```
Write C++ code using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons): [image not reproduced]
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following
constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17
cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as
78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are
already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a)
solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle,
consider the bottom right block, top right cell. Because of other numbers in the same row, column, or
```

```
grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion,
using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
- That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search,
in which we exhaustively try possibilities until we find a solution. This will be slower to run but is
easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a
few milliseconds.)
- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify
an empty cell and deduce a list of candidates for what might go into that cell. If there are no such
values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made
a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the
first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled
(i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and
so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates
report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make
an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find
something else to try, and so on. If a solution exists, we are guaranteed to find it, because we
exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits
already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should
print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

---

## Input clarification

The five puzzle files (puzzle1.txt through puzzle5.txt) and solution1.txt are
in the same folder as this file.

The program must find its puzzle files in either of two ways:

- If no command-line arguments are given, solve puzzle1.txt, puzzle2.txt,
  puzzle3.txt, puzzle4.txt and puzzle5.txt, in that order, from the current
  working directory: `./program`
- If one or more filenames are given as command-line arguments, solve those
  files instead, in the order given: `./program puzzle3.txt`

All output goes to standard output. The program exits with status 0 when it
completes normally.
```

2d. The programs they generated

Claude Haiku 4.5 (`Claude/iterations/00-raw.cpp`)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class Sudoku {
private:
    std::vector<std::vector<int>> grid;
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;

    bool isValid(int row, int col, int val) const {
        // Check row
        for (int c = 0; c < SIZE; c++) {
            if (grid[row][c] == val) return false;
        }
```

```
        // Check column
        for (int r = 0; r < SIZE; r++) {
            if (grid[r][col] == val) return false;
        }

        // Check 3x3 block
        int blockRow = (row / BLOCK_SIZE) * BLOCK_SIZE;
        int blockCol = (col / BLOCK_SIZE) * BLOCK_SIZE;
        for (int r = blockRow; r < blockRow + BLOCK_SIZE; r++) {
            for (int c = blockCol; c < blockCol + BLOCK_SIZE; c++) {
                if (grid[r][c] == val) return false;
            }
        }

        return true;
    }

    std::vector<int> getCandidates(int row, int col) const {
        std::vector<int> candidates;
        if (grid[row][col] != 0) return candidates;

        for (int val = 1; val <= SIZE; val++) {
            if (isValid(row, col, val)) {
                candidates.push_back(val);
            }
        }
        return candidates;
    }

    bool solve(bool& foundSolution) {
        // Find next empty cell
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                if (grid[r][c] == 0) {
                    std::vector<int> candidates = getCandidates(r, c);

                    if (candidates.empty()) {
                        return false;
                    }

                    bool anyValid = false;
                    for (int val : candidates) {
                        grid[r][c] = val;

                        if (solve(foundSolution)) {
                            anyValid = true;
                        }

                        grid[r][c] = 0;
                    }
                    return anyValid;
                }
            }
        }

        // All cells filled - found a solution
        foundSolution = true;
        return true;
    }

public:
    Sudoku() : grid(SIZE, std::vector<int>(SIZE, 0)) {}

    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int r = 0; r < SIZE; r++) {
```

```
            for (int c = 0; c < SIZE; c++) {
                char ch;
                file >> ch;
                if (ch == '_') {
                    grid[r][c] = 0;
                } else {
                    grid[r][c] = ch - '0';
                }
            }
        }

        file.close();
        return true;
    }

    void print() const {
        for (int r = 0; r < SIZE; r++) {
            if (r % BLOCK_SIZE == 0 && r != 0) {
                std::cout << "------+-------+------\n";
            }
            for (int c = 0; c < SIZE; c++) {
                if (c % BLOCK_SIZE == 0 && c != 0) {
                    std::cout << "| ";
                }
                if (grid[r][c] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << grid[r][c] << " ";
                }
            }
            std::cout << "\n";
        }
    }

    Sudoku copy() const {
        Sudoku newPuzzle;
        newPuzzle.grid = this->grid;
        return newPuzzle;
    }

    bool solve() {
        bool foundSolution = false;
        solve(foundSolution);
        return foundSolution;
    }
};

int main(int argc, char* argv[]) {
    std::vector<std::string> files;

    if (argc == 1) {
        files = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    } else {
        for (int i = 1; i < argc; i++) {
            files.push_back(argv[i]);
        }
    }

    for (const auto& filename : files) {
        std::cout << "Puzzle: " << filename << "\n";

        Sudoku puzzle;
        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Could not open file: " << filename << "\n\n";
            continue;
        }

        std::cout << "Original:\n";
        puzzle.print();
        std::cout << "\n";
```

```
        if (puzzle.solve()) {
            std::cout << "Solution:\n";
            puzzle.print();
        } else {
            std::cout << "No solution found\n";
        }

        std::cout << "\n";
    }

    return 0;
}
```

GPT-6 Luna (`Codex/iterations/00-raw.cpp`)

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class Sudoku {
private:
    std::array<std::array<int, 9>, 9> cells{};
    std::vector<std::array<std::array<int, 9>, 9>> solutions;
    static constexpr std::size_t solutionLimit = 100000;

    bool allowed(int row, int column, int value) const {
        for (int i = 0; i < 9; ++i) {
            if (cells[row][i] == value || cells[i][column] == value)
                return false;
        }

        int blockRow = (row / 3) * 3;
        int blockColumn = (column / 3) * 3;
        for (int r = blockRow; r < blockRow + 3; ++r) {
            for (int c = blockColumn; c < blockColumn + 3; ++c) {
                if (cells[r][c] == value)
                    return false;
            }
        }
        return true;
    }

    void search() {
        if (solutions.size() >= solutionLimit)
            return;

        int row = -1;
        int column = -1;
        for (int r = 0; r < 9 && row == -1; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (cells[r][c] == 0) {
                    row = r;
                    column = c;
                    break;
                }
            }
        }

        if (row == -1) {
            solutions.push_back(cells);
            return;
        }

        for (int value = 1; value <= 9; ++value) {
            if (allowed(row, column, value)) {
                cells[row][column] = value;
                search();
```

```
                cells[row][column] = 0;
                if (solutions.size() >= solutionLimit)
                    return;
            }
        }
    }

public:
    bool load(const std::string& filename) {
        std::ifstream input(filename);
        if (!input)
            return false;

        std::string line;
        int row = 0;
        while (row < 9 && std::getline(input, line)) {
            int column = 0;
            for (char ch : line) {
                if (ch == ' ' || ch == '\t' || ch == '\r' || ch == '|')
                    continue;
                if (ch >= '1' && ch <= '9') {
                    cells[row][column++] = ch - '0';
                } else if (ch == '_') {
                    cells[row][column++] = 0;
                }
                if (column == 9)
                    break;
            }
            if (column == 9)
                ++row;
            else if (column != 0)
                return false;
        }
        if (row != 9)
            return false;

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                int value = cells[r][c];
                if (value != 0) {
                    cells[r][c] = 0;
                    bool valid = allowed(r, c, value);
                    cells[r][c] = value;
                    if (!valid)
                        return true; // Search will correctly report no solutions.
                }
            }
        }
        return true;
    }

    void solve() {
        solutions.clear();
        search();
    }

    void printPuzzle() const {
        printGrid(cells);
    }

    void printSolutions() const {
        if (solutions.empty()) {
            std::cout << "No solution found\n";
            return;
        }
        for (std::size_t i = 0; i < solutions.size(); ++i) {
            if (solutions.size() > 1)
                std::cout << "Solution " << i + 1 << ":\n";
            printGrid(solutions[i]);
        }
    }
```

```
private:
    static void printGrid(const std::array<std::array<int, 9>, 9>& grid) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (c != 0)
                    std::cout << ' ';
                std::cout << (grid[r][c] == 0 ? '_' : char('0' + grid[r][c]));
            }
            std::cout << '\n';
        }
    }
};

class SudokuProgram {
private:
    std::vector<std::string> filenames;

public:
    SudokuProgram(int argc, char* argv[]) {
        if (argc > 1) {
            for (int i = 1; i < argc; ++i)
                filenames.emplace_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                filenames.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    int run() const {
        for (std::size_t i = 0; i < filenames.size(); ++i) {
            const std::string& filename = filenames[i];
            if (i != 0)
                std::cout << '\n';
            std::cout << filename << '\n';

            Sudoku puzzle;
            if (!puzzle.load(filename)) {
                std::cout << "Unable to read puzzle\n";
                continue;
            }
            puzzle.printPuzzle();
            puzzle.solve();
            puzzle.printSolutions();
        }
        return 0;
    }
};

int main(int argc, char* argv[]) {
    SudokuProgram program(argc, argv);
    return program.run();
}
```

2e. My comparison

Both programs use one recursive function that finds the first empty cell, tries the digits 1 to 9 that fit, calls itself, and puts the cell back to blank when it backs up. That is the depth-first search with backtracking the assignment describes. Claude put everything in one `Sudoku` class and loops over the files in `main`. Codex used a `Sudoku` class plus a `SudokuProgram` class that handles the files, so its `main` is only 4 lines.

Correctness

Both compiled with no errors or warnings, even with the extra warning flags turned on. The big difference showed up as soon as I ran them on the five puzzles. I checked every grid each program printed against an independent solution counter written in Python.

Puzzle	Correct answer	Claude	Codex
puzzle1	1 solution	Printed the unsolved puzzle	Correct, matches `solution1.txt`
puzzle2	1 solution	Printed the unsolved puzzle	Correct
puzzle3	1 solution	Printed the unsolved puzzle	Correct
puzzle4	No solution	“No solution found”	“No solution found”
puzzle5	2 solutions	Printed the unsolved puzzle	Printed both solutions

Claude’s program does find the solutions, but when it backs up it sets every cell back to blank, and it never saves a copy of the finished grid. So by the time it prints “Solution:”, the grid is back to the original puzzle. It also only prints one grid, so it could never show both answers to puzzle 5. Its own report said it printed “only the first solution,” which was not true either.

Then I ran both programs through my test scripts on 34 random puzzles, a puzzle with 196 solutions, one with 29,040 solutions, and a well-known hard puzzle. Codex matched the solution counter on every one. Claude matched none of them. Both handled the course files’ Windows line endings and the same files saved with Mac/Linux line endings.

The tests also showed a few other differences:

Test	Claude	Codex
Missing file	Says it could not open the file	Says “Unable to read puzzle”
File with 8 rows, a letter, or nothing in it	Treats it as a puzzle and prints junk numbers like 72 and -48	Says “Unable to read puzzle”
Completely blank grid (billions of solutions)	Still running after 10 seconds	Stops after 100,000 solutions without saying it stopped

The assignment only promises puzzles in the given format, so the bad-file tests are extra. The blank grid matters a little more: Codex’s limit means it does not really “print them all”, but it does not tell you that. Claude never prints anything on the blank grid; it just keeps searching silently until it is stopped. The full test results are in `evidence/run00-bench-local.txt`. I also ran the same tests on the cycle server (cycle1, x86-64, g++ 11.4), and every correctness result came out the same. The only difference was the blank grid: cycle is slower, so Codex was still printing (about 50,000 solutions in) when the 10-second limit hit. Those results are in `evidence/run00-bench-cycle.txt`.

Execution time

Brute force can get slow fast. Each blank cell can have up to 9 choices, so in the worst case every extra blank can multiply the work by up to 9. That is **O(9^b)** for **b** blank cells. In practice the rules cut most of those choices off early. Both programs search in exactly the same order, and neither one stops after finding a solution, so they do the same amount of work. On the hard puzzle, both made 2,068,781 recursive calls.

I measured these CPU times in a Linux VM on my Mac, using the plain `g++` build. One run of the five course puzzles was too quick for the timer, so I used 200 runs to get the per-run times. The larger tests show the best of 7 runs.

Test	Claude	Codex
All five course puzzles	about 1.05 ms	about 1.10 ms
29,040-solution puzzle	0.52 s	0.60 s
Hard puzzle (1 solution)	0.82 s	0.89 s

In this plain build, Claude was a little faster, but the times were within about 15%. I would not choose a program based on this small gap. I did find a reason the timing might change: Claude makes a new list of possible digits at each step. On the hard puzzle, it asked for memory 2,048,819 times; Codex did that only 4 times. When I compiled with `-O2`, which lets the compiler speed up the code, Codex finished in 0.08 s and Claude in 0.13 s. That separate test is recorded in `evidence/timing-cause.txt`. On the cycle server everything took about four times longer, but the order stayed the same: on the hard puzzle Claude took 3.46 s and Codex took 4.24 s.

Space complexity

Both programs keep one 9x9 grid and go one level deeper for each blank cell they fill, so there are never more than 82 levels. Each level is small: 128 bytes for Claude and 80 bytes for Codex. That part is basically constant.

The difference is that Codex saves every solution it finds before printing. Each saved solution is 324 bytes, so its memory grows with the number of solutions: twice as many solutions means about twice as much storage. That is **O(s)** space for **s** solutions. For the five course puzzles both peaked at 2,816 KB. For the 29,040-solution puzzle, Claude stayed at 2,816 KB and Codex used 13,044 KB. On the blank grid, Codex reached 44,248 KB at its 100,000-solution limit. Claude uses less memory, but only because it never keeps the answers it is supposed to print.

Maintainability

Neither program has the required opening comment with my name, description, inputs, outputs, and sources. Claude has 5 comment lines and Codex has 1, so both need real commenting.

Claude’s file is easier to read line by line, and its functions are small. But it has a `copy()` method it never uses, which is exactly the piece that would have saved the answer. Codex is organized better: file handling, solving, and printing are each in their own place, and `main` is

4 lines. Its `load` function is the weak spot. On the lecture's complexity measure (how many decisions are packed into one function), `load` scored 20, over the limit of 15. Claude's highest was 8.

2f. Which program I chose

I chose Codex's program. The main reason is correctness. Codex got all 5 course puzzles right, matched the solution counter on 34 of 34 random puzzles, and printed exactly 196 and 29,040 solutions on the bigger tests. Claude got 1 of 5 (only puzzle 4, the one with no answer) and 0 of 34, because it never saves the finished grid. Speed was about even (within about 15% on the plain build, and Codex was faster with `-O2`), so time did not decide it. Codex used more memory on puzzles with thousands of solutions, but I could fix that without touching the search itself.

The structure mattered too. In *A Philosophy of Software Design*, Ousterhout says good modules are "deep": a simple interface with a lot of work hidden behind it. Codex's `SudokuProgram` is like that. `main` just creates it and calls `run()`. Claude's program is shallow: most of the work is in a 36-line `main`, and solving and saving the answer are tangled together with nothing protecting the saved solution. That is exactly where its bug is, so fixing Claude would have meant rewriting the core of the program. Codex's one weak spot was the opposite problem: `load()` packed too much into one function (complexity 20). That was the first thing I split up.

2g. How I improved it

ChatGPT (GPT-6.1 Sol) helped me revise the code and comments, and Claude Opus 5.5 helped with the first round of changes. Both are credited in the prolog. These are the changes from Codex's original to my final `Assignment4.cpp`:

- **Print each solution as soon as it is found** instead of saving every grid in a list first. This was the biggest space improvement. Memory no longer grows with the number of solutions.
- **Removed the hidden 100,000-solution limit.** The assignment says to print every solution, and Codex quietly stopped at 100,000 without telling the user. The count is now kept as text digits, so it can never overflow either.
- **Split up `load()`.** It had complexity 20, over the limit of 15. It is now `load()` (3), `readGrid()` (6), `readRow()` (7) and `checkClues()` (5). I also pulled "find the first empty cell" out of `search()` into `findEmpty()`, so `search()` went from 10 to 4. The highest complexity in the program is now 7, and the average dropped from 5.7 to about 4.1.
- **Stricter file reading.** Each cell must be one character, and extra rows or extra cells are rejected. A file that fails to load leaves the puzzle unchanged.
- **Skip the search when two clues already break a rule**, since no solution can exist.
- **Clearer names.** The 9x9 grid type is now called `Grid` instead of writing `std::array<std::array<int, 9>, 9>` everywhere. New variables have names that say what they hold, like `cluesClash`, `solutionCount`, `nextRow` and `token`.

- **Comments and prolog.** Codex had 1 comment line and no prolog. I rewrote the comments so every line is explained (185 comment lines), added a block comment above each function saying what it does and why, and added the full prolog with revisions and collaborators.
- **Output:** every solution is numbered, and each puzzle ends with “Solutions found: N” (like `solution1.txt`), or “No solution found”.

Before and after, using the same test scripts on the same x86-64 Linux machine (g++ 13.3, plain build, best of 3 runs):

Metric	Codex original	My final version
Course puzzles correct	5 of 5	5 of 5 (also checked on cycle1)
196- and 29,040-solution puzzles	correct	correct
Blank grid	stops at 100,000 solutions without saying so	keeps printing every solution
Peak memory, 29,040-solution puzzle	13,836 KB	3,752 KB
Peak memory, blank grid	45,036 KB	3,456 KB
Time, 29,040-solution puzzle	0.99 s	0.97 s
Time, hard puzzle	1.48 s	1.44 s
Highest complexity (limit 15)	20	7
Average complexity per function	5.7	about 4.1
Comment lines	1	185
Prolog	none	yes

The time is about the same, which makes sense because the search itself did the same work in both. The real gains are in memory (about 3.7 times less on the 29,040-solution puzzle, and it no longer grows), in correctness (it really prints all solutions now), and in maintainability (no function over the complexity limit, and every line commented). I compiled the final file on cycle1 with `g++ Assignment4.cpp -o Assignment4.exe` (g++ 11.4, x86-64). It printed the right result for all five puzzles, and puzzle 1 matches `solution1.txt`.